

Reputation-Based Leader Election under Partial Synchrony: Towards a Protocol-Independent Abstraction with Enhanced Guarantees

Xuyang Liu^{†,‡}, Zijian Zhang^{†,*}, Zhen Li[†], Jiahang Sun[§], Jiamou Liu[‡], Peng Jiang^{†,*}

[†]School of Cyberspace Science and Technology, Beijing Institute of Technology, China

[‡]School of Computer Science, The University of Auckland, New Zealand

[§]School of Computer Science, Beijing Institute of Technology, China *Corresponding author

Email: {liuxuyang, zhangzijian, zhen.li, sunjh, pengjiang}@bit.edu.cn, jiamou.liu@auckland.ac.nz

Abstract—Leader election serves a well-defined role in leader-based BFT protocols. Existing reputation-based leader election frameworks for partially synchronous BFTs suffer from either protocol-specific proofs, narrow applicability, or unbounded recovery after network stabilization, leaving an open problem. This paper presents a novel protocol-independent abstraction formalizing generic correctness properties and effectiveness guarantees for leader election under partial synchrony, enabling protocol-independent analysis and design. Building on this, we design the Sliding Window Leader Election (SWLE) mechanism. SWLE dynamically adjusts leader nominations via consensus-behavior-based reputation scores, enforcing Byzantine-cost amplification. We demonstrate SWLE introduces minimal extra overhead to the base protocol and prove it satisfies all abstraction properties and provides superior effectiveness. We show, with a 16-server deployment across 4 different regions in northern China, SWLE achieves up to 4.2× higher throughput, 75% lower latency and 27% Byzantine leader frequency compared to the state-of-the-art solution under common Byzantine faults, while maintaining efficiency in fault-free scenarios.

Index Terms—distributed consensus, Byzantine fault tolerance, leader election, partial synchrony.

I. INTRODUCTION

Overview. This paper addresses limitations in existing reputation-based leader election frameworks for partially synchronous BFTs that either tie correctness to specific protocols, target narrow scenarios or protocol classes, or lack bounded recovery post-GST. We present a novel protocol-independent abstraction formalizing generic properties for theoretical analysis. Based on this, we design a novel mechanism, providing proven correctness, effectiveness (including bounded post-GST recovery), significantly outperforming state-of-the-art approach under common Byzantine faults.

History. Byzantine Fault Tolerant (BFT) state machine replication originated from Lamport et al.'s Byzantine Generals Problem [1], which formalized achieving deterministic consensus in distributed systems with malicious actors. They proved the impossibility of solving consensus with f traitors among $3f$ generals, while providing a solution for $3f + 1$ replicas under synchrony with high communication complexity. Fischer et al. [2] later showed deterministic consensus is impossible in asynchronous settings even with a single fault, necessitating

randomness for liveness. Ben-Or [3] established the fundamental $(n - 1)/3$ resilience bound under asynchrony, requiring at least $3f + 1$ replicas to tolerate f Byzantine faults.

Partial Synchrony. In 1988, Dwork et al. proposed the partially synchronous model [4], introducing the Global Stabilization Time (GST) - an unknown finite time after which network becomes synchronous. This model enables BFT protocols to ensure safety during all periods and liveness after GST, provided new directions for designing BFT protocols.

Leader-Based Paradigm. Leader-based paradigms emerged as dominant approach in consensus development: one replica serves as leader/primary to coordinate consensus while others function as voters. Initially introduced for conflict resolution to ensure liveness by providing a single authority to order conflicting operations [5], the role evolved to leaders being fully responsible for initiating proposals. The typical consensus flow involves leader election, proposal dissemination, voting, and decisions based on supermajority (mainly BFT) or majority (mainly Crash Fault Tolerance (CFT)) agreement. If consensus is achieved, system commits and progresses; otherwise, it aborts and elects a new leader to restart the process.

Unique Benefits of Partially Synchronous & Leader-Based BFTs. Partial synchrony occupies an intermediate position between full synchrony and asynchrony, closely modelling real-world networks with bounded delays and inherent uncertainties. It organizes consensus into views [6], providing an elegant design framework balancing safety and liveness guarantees. The leader-based paradigm further amplifies these benefits through coordinated leadership, delivering reduced communication complexity, simplified protocol design, increased efficiency. Consequently, these protocols become impactful and form core infrastructure of permissioned blockchains [7].

Leader Election under Partial Synchrony: Challenges, Existing Solutions, and the Open Problem. Classical BFTs [6], [8] employ stable leaders until failure which is then replaced by designated successors. While blockchain-inspired chain-based BFTs [9], [10], [11] introduce leader rotation to address chain quality in trustless environments where replicas should fairly share responsibilities. Under leader rotation, replicas alternate as leaders following predefined rules, en-

sure that correct replicas lead infinitely with fixed proportions as view increases, but unfortunately extending same guarantee to Byzantine replicas. Since leaders control proposals, Byzantine leaders can severely disrupt consensus. Most fundamentally, they can simply drop messages until timeout-triggered rotation occurs (cost-free), causing considerable performance degradation as timeouts typically far exceed normal commit latencies.

Several approaches have emerged to mitigate Byzantine leader impact, targeting transaction ordering [12], [13], communication protocols [14], [15], or imposing computational costs through hash-based elections [16]. Recent research has explored reputation-based election approaches [17], [18], [19], [20] where reputation-based signifies relying solely on available information during consensus process to determine leaders, offering self-contained solutions that avoid external dependencies. However, state-of-the-art solutions primarily focus on crash-only scenarios with limited practical significance, and none simultaneously achieve bounded recovery after GST, protocol-independence and general applicability to partially synchronous BFTs, leaving this as an open problem.

A. Our Contributions.

To address these limitations, this paper presents a comprehensive solution spanning theoretical foundations, mechanism design, and practical implementation.

Theoretical foundations. We first establish a generic abstraction framework, defining three core properties that captures the essential correctness and effectiveness guarantees of leader election mechanisms in partially synchronous settings. This decouples correctness proofs from specific protocols, enabling protocol-independent analysis and design.

Mechanism design. Building upon this foundation, we design the Sliding Window Leader Election (SWLE) mechanism that dynamically adjusts leader nominations based on replica behavior. The core design philosophy of SWLE centers on:

- Determining Leadership eligibility by accumulated reputation reflecting consensus participation and behavior quality.
- Separating election from current view by finalizing future leaders (sufficiently in advance to maintain liveness post-GST) through authenticated consensus decisions, eliminating race conditions between election and view progression.
- Maintaining a fixed-size window for future leaders and triggering fair, systematic elections universally.
- Enforcing Byzantine-cost amplification and introducing minimal extra overhead to the base protocol.

It integrates with any BFT satisfying two prerequisites (align with core properties of most partially synchronous BFTs). We prove SWLE satisfies our abstraction and provides superior effectiveness compared to conventional approaches while maintaining bounded recovery after network stabilization.

Practical Implementation. SWLE is implemented as a lightweight Rust module (~ 400 LoC) and its integration requires minimal modifications (e.g., ~ 300 LoC in non-chained HotStuff [9]). We show, with a 16-server deployment across 4 regions in northern China, SWLE achieves up to $4.2\times$ higher throughput, 75% lower latency and 27% Byzantine

leader frequency under common Byzantine faults compared to state-of-the-art solution ABSE [18] and Hotstuff's original election mechanism, ensures bounded recovery post-GST, while maintaining minor extra overhead in fault-free scenarios.

Thus, this paper makes the following contributions:

- A protocol-independent abstraction formalizing correctness and effectiveness for reputation-based leader election.
- SWLE mechanism build upon the abstraction, with formal proofs of satisfying all properties under partial synchrony.
- An open-source, lightweight implementation demonstrating seamless integration and superior performance.

II. RELATED WORK

Fault Tolerant Consensus. Fault-tolerant consensus protocols can be primarily categorized into CFT and BFT, with CFT addressing benign failures (e.g., Paxos [21]). BFT addresses arbitrary failures and can be categorized into synchronous [22], [10], partially synchronous [6], [9], [23], and asynchronous [24], [25] variants based on network assumptions. In this work, we focus on partially synchronous BFTs, widely adopted as core building block for permissioned blockchains [7].

Leader in BFTs. Most BFTs employ a leader-based model. While leader roles vary across protocols [6], [26], [27], they fundamentally coordinate ordering and facilitate agreement among replicas. Several studies [28], [29] identified leaders' significant influence on consensus performance, with some exploring leaderless alternatives [30], [31]. In this work, we focus on enhancing leader-based protocols through reputation mechanisms while preserving coordination advantages.

Chain-based BFTs. Blockchains spur innovations like chain-based BFTs [9], [10], [11], which introduce leader rotation for chain quality, and employ linear block structures for streamlined message patterns and decision pipelining. A significant advancement is HotStuff [9], which first develops a basic non-chained version combining collective signatures [8] and block locking [32], then extends to a chained variant via pipelining. While chain-based BFTs share core safety with classical designs, liveness exhibit subtle distinctions under partial synchrony. Due to space constraints, we establish SWLE's properties under classic pattern for our fundamental approach clarity, noting seamless extension to chain-based variants (similar to extending BFT core correctness properties).

Existing Approaches Mitigating Byzantine Leader Impact.

A wealth of research targets leader vulnerabilities in BFTs. Approaches like Aequitas [12] and Themis [13] mitigate adversarial ordering by guaranteeing transaction order-fairness. Prosecutor [16] imposes computational burdens via hash-based election campaigns. Communication-level solutions [14], [15] leverage balanced reliable broadcast or verifiable information dispersal to eliminate leaders as communication hubs during broadcasts—though often at excessive overhead.

Recent reputation-based approaches offer self-contained alternatives. PrestigeBFT [17] uses reputation-based computational penalties for active view changes. ABSE [18] introduces an adaptive baseline score-based election framework but lacks protocol-independence and bounded post-GST recovery.

Carousel [19] uses on-chain data for local leader determination without consensus dependency, but focuses primarily on crash-only scenarios in chain-based BFTs. While Tsimos et al. [20] extend its concepts to DAG-based BFTs, it creates framework divergence rather than unification. We provide detailed comparisons between our abstraction and state-of-the-art frameworks in Section IV-C.

III. SYSTEM MODEL

We consider a state machine replication (SMR) [33] system of n replicas denoted as $\mathcal{R} = \{1, 2, \dots, n\}$. We assume a Public Key Infrastructure as the cryptographic primitive where each replica i has a fixed public-private key pair (pk_i, sk_i) during protocol execution, using digital signatures to authenticate messages, secure against computationally bounded adversaries. The system operates under partial synchrony [4]: an unknown Global Stabilization Time (GST) exists such that after GST, all messages between correct replicas are delivered within a known bounded delay Δ . Before GST, message delays are unbounded, though communication channels remain reliable (eventual message delivery) and point-to-point.

We consider a Byzantine adversary controlling up to $f < n/3$ replicas (trusted hardware-dependent [34] protocols may achieve $(n-1)/2$ resilience; for simplicity, we omit this here and define $n = 3f+1$), which may behave arbitrarily: collude, produce arbitrary values, eavesdrop, selectively delay or omit messages, and send conflicting information, but can not compromise the cryptographic primitives, prevent point-to-point communications between correct replicas, or indefinitely delay GST. Correct replicas strictly follow the protocol specification.

IV. LEADER ELECTION ABSTRACTION

Leader election serves a well-defined role in leader-based BFT protocols. In this work, we provide a generic foundation for designing reputation-based leader election mechanisms under partial synchrony, where reputation-based signifies leader determination relies solely on information available during consensus process or consensus-generated on-chain information. Instead of protocol-specific designs, we emphasize abstracting election patterns prevalent in most partially synchronous protocols. We then demonstrate how it enables design of a novel mechanism to replace conventional approaches.

A. Leader-based BFTs under Partial Synchrony

Generally speaking, any leader-based BFT includes at least a round-based normal state protocol. Partially synchronous BFTs additionally rely on view changes (with a form of leader rotations) for liveness and therefore proceed in views. For self-containment, before abstracting properties, we provide a brief high-level description of the workflow of such protocols:

The system progresses through sequential views $v = 1, 2, \dots$, where during v a replica in $\mathcal{R}$, chosen through an election process, serves as the leader to drive the consensus protocol for one consensus round (explicitly assigned a unique sequence number in some protocols [6]) or more (some protocols separate data transition from consensus [27] or create

multiple instances [35]; for such protocols, we consider only one instance by default). Each consensus round comprises one or more phases, which can be broadly categorized into:

- 1) **Dissemination phases:** where the leader broadcasts information (proposals/updates) to all replicas. For brevity, we will refer to these collectively as proposals.
- 2) **Voting phases:** where replicas vote on the proposals, with most protocols requiring a supermajority (i.e., $2f+1$ votes from distinct replicas) for acceptance.

A view change is triggered when a leader appears faulty (i.e., timer expiration). Some protocols (without affecting timeout-based changes) also include auxiliary strategies (e.g., triggered periodically [36]), or embed view change mechanisms within normal state protocol, causing leaders to rotate with consensus rounds [9]. The two most common leader election strategies include rolling-based selection [11], [9] (round-robin fashion, e.g., leader of v is replica $v \bmod n$) and predefined leaders per view before consensus begins [6], [27].

Asynchronous BFTs, however, do not have view change mechanisms. Furthermore, the FLP result [2] demonstrates the fault tolerance problem cannot be solved deterministically in asynchronous settings, requiring additional randomness during leader election for liveness (typically via additional building blocks, e.g., common coins [37]). Due to such significant differences, we do not jointly consider asynchronous assumptions, which could yield properties either too broad with model-specific redundancies, or too complex for convenient mechanism design. We aim for properties that are concise, intuitive, and can effectively guide new mechanisms design.

B. Property Specification

In our formalization, we abstract generic correctness properties for leader election mechanisms under partial synchrony, some of which are derived from existing protocol designs. Our core objective is to enable a single universal correctness proof for any mechanism satisfying these properties, eliminating the need for protocol-specific proofs. Let $\text{LEADER}_j(v)$ denote replica j 's determination of the elected leader for view v . We first define the following fundamental properties:

Leader Uniqueness: For any view v , there do not exist two distinct replicas j and k such that both can gather a quorum of votes (i.e., $2f+1$ distinct votes) for conflicting leadership claims in v .

Timely Finalization: For any view v , every correct replica j finalizes $\text{LEADER}_j(v)$ before or upon entering v .

Leader Uniqueness is a safety property, while Timely Finalization is a liveness property (dependent on eventual synchrony after GST). Briefly, common leader election mechanisms under partial synchrony essentially fix the leader per view before consensus begins, hence easily satisfying both properties. This is, however, not the case for reputation-based leader election mechanisms, because replica reputation needs to be established during protocol execution – leader selection strategies needs to change dynamically as consensus progresses. Nevertheless, it is also important to note a leader election mechanism should focus solely on the process of electing leaders and **should not**

alter the leader's functional role in the protocol (e.g., Byzantine leader tolerance mechanisms should remain unchanged). The key enhancement should lie in reducing the probability of Byzantine leaders being elected through reputation mechanisms, thereby decreasing the frequency of scenarios where "tolerating Byzantine leaders" becomes necessary).

While the above two properties ensure basic consistency and leader availability, they are still insufficient for designing a effective leader election mechanism. Consider the worst-case scenarios for a mechanism satisfying both properties:

- 1) The mechanism consistently elect Byzantine leaders.
- 2) Correct replicas finalize different leaders for same views (i.e. no replicas can receive $2f+1$ votes during elections).

In both cases, consensus cannot progress normally (liveness issues). This motivates an extra property to quantify a mechanism's effectiveness - measuring how well it promotes progress by electing unified correct leaders among correct replicas. This can also be used to theoretically evaluate whether it can serve as a replacement for a protocol's original mechanism

Attempt 1. An ideal enhancement to a leader election mechanism that could implement eventual perfect failure detection may be defined as: faulty replicas are permanently identified and correct replicas are never suspected (or, equivalently, for any correct replica there always exist views where it is elected among correct replicas, and no view exists where a Byzantine replica is elected). Unfortunately, a reputation-based leader election mechanism cannot achieve this definition because:

- It must allow suspected correct replicas to be unsuspected based on good behavior and hence cannot permanently isolate a Byzantine replica once the latter starts behaving well and accumulates sufficient reputation.
- Byzantine replicas can choose to behave correctly, making them indistinguishable from correct replicas.
- Under network delays or asynchronous periods, even correct replicas might occasionally fail to accumulate reputation, thus being suspected.

Attempt 2. Another choice is to define the following property:

Increased Adversarial Cost: Byzantine replicas incur higher cost of misbehavior to be elected as leaders.

This property essentially creates a tradeoff, which in turn indirectly reflects the effectiveness of a leader election mechanism, and is more feasible than Attempt 1: Byzantine replicas can no longer simply wait for their turn in the rotation without any good behavior required to eventually become leaders. For example, if Byzantine replicas need to contribute positively to consensus for a substantial period before gaining leadership eligibility, the overall system performance improves compared to scenarios where they can disrupt without such constraints.

Unfortunately, first, this property remains incomplete because it still does not consider whether correct replicas can be elected. Second, terms like "higher" are essentially comparative properties that require reference to a baseline mechanism (e.g., protocol's original mechanism), which violates our protocol-independent abstraction goal. Furthermore, such terms are too vague and cannot well measure effectiveness.

We comment that while we do not adopt Increased Adversarial Cost, we retain its core intuition. A leader election mechanism can indeed be motivated by this idea. Moreover, if a mechanism (whether reputation-based or not) preserves the original leadership eligibility (under common mechanisms) of correct replicas, then the property will naturally apply to it.

Final Attempt. Finally, we define the following property:

γ -Guarantee: A leader election mechanism satisfies the γ -Guarantee (where $\gamma > 0$) if there exist positive integers v_c and T , and a constant $\sup \leq T$, such that for any view $v > v_c$, over any sequence of T consecutive views starting from v , the expected number of views $E[C]$ where: (1) All correct replicas agree on the same elected leader, and (2) This leader is correct, satisfies: $\sup \geq E[C] \geq \gamma T$

γ -Guarantee is also a liveness property, and is slightly unconventional. It is somewhat similar to the chain quality property in Bitcoin [38], having both deterministic guarantees (addressing the liveness issues) and measurement indicators (effectiveness). We explain why $E[C]$ is bounded by a range:

Lower Bound (γT): Acknowledges that Byzantine replica behavior is variable and that perfect detection is impossible (Attempt 1). When Byzantine replicas behave correctly, they become indistinguishable from correct replicas under reputation-based leader election mechanisms and cannot be identified. It is hoped that this situation serves as the lower bound for a leader election mechanism: Byzantine replicas cannot further reduce the effectiveness guarantee (i.e. reduce $E[C]$ below γT) through collusion or manipulation. It is also hoped this compels Byzantine replicas to maintain "good behavior" to the greatest extent possible¹ (similar to correct replicas) to remain eligible for leadership - indirectly increasing their cost of misbehavior as discussed in Attempt 2, which has a positive impact on system overall performance.

Upper Bound ($\sup$): The bound represents the best-case effectiveness under optimal conditions. Different mechanisms may implement different scenarios when achieving it (e.g., consider the case where all faults are crash faults - a common approach, although in practice, it has limited effect on a BFT system).

We now explain why we consider correct replicas collectively rather than individually: Correct replicas may exhibit heterogeneous capabilities (e.g., network latency). Mechanisms should permit leadership concentration on higher-performing correct replicas (e.g., those with lower average message delay) to enhance system throughput.

Leader Uniqueness combined with Timely Finalization and γ -Guarantee guarantees the correctness of a leader election mechanism, while its effectiveness needs to be measured according to parameters (theoretically). A mechanism can be correct (γ is small) but ineffective. Standard protocols with common mechanisms (e.g., round-robin with $3f+1$ replicas) typically achieve: ($v_c = 0, T = n, \gamma = \frac{2f+1}{n}, \sup = \gamma T$).

¹We abandon the quantification of Byzantine replica misbehavior costs to avoid referencing other mechanisms in the property (avoiding protocol-specific tendencies), i.e., it can only serve as an informal indicator. Similarly to the aforementioned, it is also suitable as motivation for new mechanism design.

Reputation-based mechanisms usually require $v_c > 0$ for reputation initialization, but target a higher sup or γ .

C. Comparison with State-of-the-art Works

Our specification shares similarities with Cohen et al.'s Leader-Aware SMR framework (Carousel) [19] and Liu et al.'s ABSE framework [18]: All aim to provide a generic foundation for optimizing leader election. We and Carousel consider partial synchrony, while ABSE additionally considers asynchrony. Furthermore, all focus on designing new mechanisms via consensus-reputation-based approaches that rely solely on information available during consensus process - a self-contained design philosophy that promotes simplicity and avoids external dependencies. Our specification for correctness properties, however, differs significantly from both.

ABSE categorizes leader election into three (two consistency one conflict) scenarios, with distinct requirements for each to ensure correctness. First, while covering both partial synchrony and asynchrony, ABSE's requirements bifurcate based on network assumptions, yielding a specification neither concise nor intuitive. Second, ABSE ties correctness to specific protocols - a mechanism from ABSE cannot be proven correct in isolation but rather needs to be proven in conjunction with the target protocol after integration, i.e., each integration with a new protocol requires a new set of protocol-specific proofs. Additionally, ABSE's properties focus solely on correctness without quantifying effectiveness, hindering theoretical evaluation. Finally, conflict scenario correctness is established on rolling back original protocol flow (i.e., without ABSE), further undermining its protocol-independence characteristic. Overall, ABSE does not achieve true generality.

Carousel focuses exclusively on chain-based BFTs, narrowing its applicability compared to our specification. More critically, Carousel's specification shifts focus from leader election guarantees to block commit properties. For example, its Chain Quality property states: "For any block B committed by a correct replica, the proportion of Byzantine blocks on B 's implied chain is bounded", which provides weaker guarantees than γ -Guarantee because Byzantine replicas can do more than just propose Byzantine blocks after being elected. Most simply, it fails to capture scenario where Byzantine leaders do not produce bad blocks but block progress by dropping all messages sent by correct replicas. Such a scenario can cause considerable performance degradation without violating Chain Quality. Furthermore, the focus on block commitment also weakens Carousel's protocol-independent aspects. Besides Chain-quality, Carousel also contains properties explicitly tied to specific execution scenarios (e.g., "crash-only executions"). Although Tsimos et al. [20] extend Carousel's concepts to DAG-based BFTs, their formalization is explicitly restricted to DAG-based BFTs, creating framework divergence rather than unification, while inheriting the same specification limitations.

V. SLIDING WINDOW LEADER ELECTION (SWLE)

Following abstraction, we design SWLE. It enhances leader election by dynamically adjusting leader nominations based on

replica behavior. Its core design philosophy centers on:

- **Reputation-based candidacy:** Leadership eligibility is determined by accumulated reputation scores reflecting consensus participation and behavior quality;
- **Decoupled leader finalization:** Separating leader election from current consensus rounds by finalizing future leaders (sufficiently in advance to maintain liveness after GST) through authenticated consensus decisions;
- **Sliding window management:** Maintaining a fixed-size window of views for future leaders while providing fairness through systematic election triggering across all replicas;
- **Byzantine-cost amplification:** Forcing Byzantine replicas to maintain good behavior to acquire leadership.

SWLE is designed as a generic enhancement operating atop any partially synchronous leader-based BFT that satisfies the following two fundamental Prerequisites (align with core correctness properties of most partially synchronous BFTs):

Prerequisite 1 (Safety): For any consensus round r , if two distinct proposals $prop_1$ and $prop_2$ both originate from r , they cannot be both finalized, each by a correct replica.

Prerequisite 2 (Liveness)²: After GST, there exists a bounded interval T_b such that if all correct replicas remain in consensus round r during T_b , and the leader for r is correct, then a decision is reached (i.e., a proposal is finalized) among all correct replicas within r .

It is always possible to set a timer (waiting interval) such that all correct replicas will eventually have at least T_b overlap in common. Therefore, Prerequisite 2 can be equivalently stated as: After GST, if the leader for r is correct, then a decision can be reached among all correct replicas within r .

The liveness guarantee admits an equivalent formulation: After GST, correct leadership in r implies bounded-time finalization. More generally, it can be extended to state that for any sequence of T consecutive rounds with at least T_c rounds with correct leaders ($T \geq T_c > 0$), the decision (among all correct replicas) latency T_f (measured in consensus rounds) is bounded (also, as noted, for pipelined protocols, $T_c > 0$ alone may be insufficient. We also omit detailed discussion here).

SWLE leverages these protocol Prerequisites to provide enhanced guarantees. Any protocol satisfying Prerequisites 1-2 can integrate SWLE while preserving correctness. Furthermore, we prove that under SWLE, after GST, there exists a view v_c beyond which the decision latency satisfies $T_f < n$.

A. Mechanism Design

SWLE maintains leader assignments across a sliding window of views. We define $T_z = \lceil T_f/n \rceil \cdot n$ as the sliding window size. Each view corresponds to exactly one consensus round. A view progresses to the next when either: (1) the current view's leader successfully completes the consensus round, or (2) the maximum waiting interval for a consensus round expires (timeout occurs).

²Note that for chain-based protocols, where a proposal may span multiple consecutive rounds (i.e., traverse multiple leaders), it may require leaders are correct for cr (the number of rounds needed to finalize a proposal) consecutive rounds. However, for simplicity, we omit detailed discussion of chain-based protocols in this work to maintain clarity of our fundamental approach.

Each replica j maintains a LEADERLIST data structure of length $T_z + 2n$ (covering views v_{curr} to $v_{\text{curr}} + T_z + 2n - 1$, where v_{curr} is the current view) which stores leader information for consecutive views. Initially, LEADERLIST covers views 1 to $T_z + 2n$ and is partitioned into two segments:

- For v in 1 to $T_z + n$: The **Elected Leader** ($\text{LEADER}_j(v)$) and the **Initial Leader** ($\text{iLEADER}_j(v)$) of v are both initialized as $v \bmod n$.
- For v in $T_z + n + 1$ to $T_z + 2n$: $\text{LEADER}_j(v) = \text{null}$, $\text{iLEADER}_j(v) = v \bmod n$

When entering v , $\text{LEADER}_j(v)$ and $\text{iLEADER}_j(v)$ are no longer modified. j will first determine either $\text{LEADER}_j(v)$ if $\text{LEADER}_j(v) \neq \text{null}$, or $\text{iLEADER}_j(v)$ otherwise as v 's leader. If a replica k gathers $2f + 1$ valid votes for leadership claims (i.e., proposes a proposal containing a valid leader certificate, see Algorithm 2) in v , j will then determine k . On completing v , j remove v from LEADERLIST, and then extends it by initializing $v_{\text{new}} = v + T_z + 2n$ such that $\text{LEADER}_j(v_{\text{new}}) = \text{null}$ and $\text{iLEADER}_j(v_{\text{new}}) = v_{\text{new}} \bmod n$ (obviously, for any view, all correct replicas will have the same initial leader).

SWLE tracks replica behavior during consensus. Each j maintains a local reputation matrix $S_j \in \mathbb{R}^{n \times 1}$ where $S_j[i] \geq 0$ represents j 's assessment of replica i 's reputation score. Table V-A summarizes the scoring mechanism employed by SWLE. For j , Score_j is updated according to these rules:

TABLE I: SWLE REPUTATION SCORING RULES

Rule	Trigger Event	Affected Replica	Score Change ^{3,4}
R1	Enter new view v	Leader ⁵ of v	⁶ $\alpha_0 = -1$
R2	View v timeout	Leader ⁵ of v	$\alpha_1 = -n$
R3	Proposal finalized	Proposal's leader	$\alpha_2 = +1$
R4	Complete view v successfully as v 's leader.	Consensus-promoting replicas ⁷ of v	$\alpha_3 = +1/n$
R5	⁴ $v \bmod \Theta = 0$ $\Theta = \max(300, 10n)$	All replicas	$\alpha_4 = +1$

³When rule Ri is triggered at replica j , for replica k in Ri.Affected_Replicas: $S_j[k] = \max(S_j[k] + \alpha_i, 0)$.

⁴Values can be adjusted (but note that this may affect SWLE's correctness and effectiveness). Initially, $S_j[k] = |\alpha_4|$ for all $j, k \in \mathcal{R}$.

⁵Determination priority: Replica that gathers $2f + 1$ valid votes for leadership claims > Elected Leader > Initial Leader.

⁶ $\forall k \in \mathcal{R}$, $S_j[k] \geq |\alpha_0|$ is required to maintain k 's leadership eligibility at j .

⁷Any replica that contributes one of the first $2f + 1$ valid votes.

Where R1 is the score deduction rule for view entry (leader accountability), R2 is the score deduction rule for a failed leadership (timeout penalty), R3 is the score reward rule for proposal finalization (success reward), and R4 is score reward rule for timely voting (participation reward). These rules ensure Byzantine leaders incur score deductions, which require subsequent good behavior to offset. R5 is the periodic normalization rule that, along with non-negative score constraint, prevents permanent marginalization and maintains system adaptability. This is particularly important in scenarios where, before GST, Byzantine replicas might persistently target certain correct replicas, causing their scores to decrease indefinitely at others, making them hard to recover leadership eligibility even after GST (i.e., the time is unbounded).

In SWLE, replicas collaboratively elect leaders for future views through authenticated consensus decisions. It employs

a sophisticated candidate selection algorithm that ensure fairness through systematic election triggering across all replicas. Replica j include additional fields l_d (determined leader for v) and $\text{Cand}_j(v_{\text{target}})$ (leadership candidates for v_{target}) in all voting messages for view v (including votes during view-change to $v + 1$). For the votes to be valid, besides standard consensus checks, l_d must be non-empty and contain a valid ID, $\text{Cand}(v_{\text{target}})$ must be non-empty, non-duplicated and contain valid IDs. The target view v_{target} is computed as follows:

$$v_{\text{target}} = \begin{cases} v + n + T_z + (\lfloor v/n \rfloor \bmod n) & \text{if Case (1)} \\ v + n + T_z & \text{if Case (2)} \\ v + T_z + (\lfloor (v-1)/n \rfloor \bmod n) & \text{otherwise} \end{cases}$$

- Case (1): $v + (\lfloor v/n \rfloor \bmod n) \leq \lceil v/n \rceil \cdot n$
- Case (2): $v/n \in \mathbb{Z} \wedge (v/n) \bmod n = 1$

Proposition 1. Let T_z be a positive integer multiple of n . For any natural number x , positive integer n , when v traverses $\{nx+1, \dots, n(x+1)\}$, v_{target} traverse $\{n(x+1)+1+T_z, \dots, n(x+2)+T_z\}$ exactly once, establishing a bijective mapping.

Proposition 1 (complete proof can be found in [39]) guarantees that the election process of each view (except the initial $T_z + n$ views) is initiated exactly once, preventing both election skips and duplicates, while ensuring fair election triggering across all replicas (i.e., every replica has equal opportunity to initiate elections for views where any given replica serves as the initial leader) thus further reducing the leverage that Byzantine replicas can exert over the election process (e.g., the chance of Byzantine replicas controlling election initiation of Byzantine replicas is reduced) and increasing the cost of manipulation for Byzantine replicas.

A replica j generates $\text{Cand}(v_{\text{target}})$ in v as in Algorithm 1:

Algorithm 1 Candidate Array Generation

```

Initialization:  $\text{Cand}_j(v_{\text{target}}) \leftarrow \emptyset$ 
1: for  $i = 0$  to  $n - 1$  do  $\text{view} \leftarrow v_{\text{target}} + i$ 
2:    $\text{candidate} \leftarrow \text{iLEADER}_j(\text{view})$ 
3:   if  $S_j[\text{candidate}] \geq |\alpha_0|$  then
4:      $\text{Cand}_j(v_{\text{target}}) \leftarrow \text{Cand}_j(v_{\text{target}}) \cup \{\text{candidate}\}$ 
5: if  $\text{Cand}_j(v_{\text{target}}) = \emptyset$  then
6:    $\triangleright$  apply periodic normalization (individually) and recompute
7:   for  $k \in \mathcal{R}$ : do  $S_j[k] \leftarrow S_j[k] + \alpha_4$ 
8:   goto line 1
9: return  $\text{Cand}_j(v_{\text{target}})$ 

```

The leader of view $v + 1$ collects $2f + 1$ valid votes containing $\text{Cand}(v_{\text{target}})$ values and the same l_d matching the leader's ID. In chain-based protocols, votes for v are inherently sent to the leader of $v + 1$. In non-chained protocols, these may be votes from standard phases for v or view-change process to $v + 1$. The leader determination process is then performed as shown in Algorithm 2.

The leader certificate $\text{LCert}_{v_{\text{target}}}$ contains both the selected leader and cryptographic proof (the original $2f + 1$ votes). A replica can verify it by performing the same computation as when creating it. $\text{LCert}_{v_{\text{target}}}$ is embedded in new proposals for $v + 1$ (as an additional field, though in chain-based protocols, this can be integrated into existing quorum certificates [9])

created by votes for the previous view), and its validity should be considered during standard proposal validation.

Algorithm 2 Candidate Selection and Proof Packaging

Required: $\mathcal{V} \triangleright \text{set of } 2f+1 \text{ valid votes containing } \text{Cand}(v_{\text{target}})$
Initialization: $\text{count}[i] \leftarrow 0$ for all $i \in \mathcal{R}$
1: $\mathcal{C} \leftarrow \{ \text{vote.Cand}_k(v_{\text{target}}) \mid \text{vote} \in \mathcal{V} \} \triangleright \text{collect candidate arrays}$
2: **for** $\text{Cand}_k(v_{\text{target}}) \in \mathcal{C}$ **do**
3: **for** each candidate $c \in \text{Cand}_k(v_{\text{target}})$ **do**
4: $\text{count}[c] \leftarrow \text{count}[c] + 1$
5: $\mathcal{W} \leftarrow \{ c \mid \text{count}[c] \geq f+1 \} \triangleright \text{find } c \text{ within } \geq f+1 \text{ votes.}$
6: **if** $\mathcal{W} \neq \emptyset$ **then**
7: Select $l \leftarrow \arg \min_{c \in \mathcal{W}} \{ v' \mid \text{iLEADER}(v') = c \wedge v' \geq v_{\text{target}} \}$
8: **else** $l \leftarrow \perp$
9: **return** $\text{LCert}_{v_{\text{target}}} \leftarrow \langle l, \mathcal{V} \rangle \triangleright \text{create a leader certificate}$

When a proposal containing $\text{LCert}_{v_{\text{target}}}$ is finalized, each replica j updates its LEADERLIST as in Algorithm 3.

Algorithm 3 LEADERLIST Update

Required: $\text{LCert}_{v_{\text{target}}}$
1: Extract $\langle l, \mathcal{V} \rangle$ from $\text{LCert}_{v_{\text{target}}}$
2: **if** $l = \perp$ **then** $\text{LEADER}_j(v_{\text{target}}) \leftarrow \text{iLEADER}_j(v_{\text{target}}) \triangleright \text{fallback}$
3: **else** $\text{LEADER}_j(v_{\text{target}}) \leftarrow l$
 $\triangleright v^*$ is the highest view lower than v_{target} whose leader is elected
4: $v^* \leftarrow \arg \max_v \{ v' < v_{\text{target}} \mid \text{LEADER}_j(v') \neq \text{null} \}$
5: **for** $v' = v^* + 1$ to $v_{\text{target}} - 1$ **do** $\triangleright \text{fill intermediate views}$
6: $\text{LEADER}_j(v') \leftarrow \text{iLEADER}_j(v')$

Lines 4-6 fill gaps using default initial leaders to ensure continuity in LEADERLIST , trying to maintain progress particularly in pre-GST or Byzantine leader scenarios. The integration of election results into consensus finalizations ensures leader updates are atomic with consensus decisions, eliminating race conditions between leader election and view progression. Meanwhile, consensus safety prevents inconsistent leader assignments within a view across correct replicas.

B. Mechanism Analysis

Due to space constraints, formal proofs of SWLE's properties are presented in [39]. We state the core theorems here.

Theorem 1. *SWLE satisfies Leader Uniqueness.*

Theorem 2. (Timely Finalization) *Under f faults out of $3f+1$ replicas, after GST, there exists a view v_c such that for any view $v > v_c$, every correct replica j finalizes $\text{LEADER}_j(v)$ and $\text{iLEADER}_j(v)$ before or upon entering v .*

Theorem 3. (γ -Guarantee) *After GST, there exists a view v_c such that SWLE satisfies the γ -Guarantee with parameters $(v_c, T = n, \gamma = \frac{2f+1}{n}, \text{sup} = n - (n/\Theta \cdot f)(1 + T_z/n))$.*

Complexity Analysis. SWLE introduces minimal overhead.

(1) Each replica independently maintains its local reputation matrix S and LEADERLIST without inter-replica communication. The leader determination, scoring and candidate selection processes (all simple computations, though the overall overhead scales with replica count) are performed locally introducing no extra Communication Complexity.

(2) Each replica j maintains a constant-size $S_j \in \mathbb{R}^{n \times 1}$ and a bounded LEADERLIST of size $T_z + 2n$ - both sizes are fixed and protocol-independent (extra Space Complexity).

(3) Voting messages carry two extra fields: l_d (leader ID, size

$O(1)$), Cand (candidate array, size $O(n)$), while proposals carry LCert . For protocols already include voting proofs or certificates from previous views, LCert can be directly integrated with only l as extra overhead (Communication Overhead).

Impact of Byzantine Replicas. S and LEADERLIST are locally maintained, Byzantine replicas cannot directly interfere with correct replicas' scoring or leader determinations. While Byzantine leaders may selectively include votes when forming LCert , valid certificates require at least $f+1$ votes from correct replicas, and this only affects their own S regarding scoring (but they already can arbitrarily manipulate their own scores regardless of rules). Pre-GST, Byzantine replicas may target certain correct replicas through message delays, reducing their scores to $< |\alpha_0|$ at others to eliminate their leadership eligibility (but liveness cannot be originally guaranteed pre-GST, i.e., they can still achieve similar effect without SWLE). However, SWLE's design ensures rapid recovery (see proofs in [39]): after GST, there exists v_c beyond which correct replicas' worst-case election probability never falls below conventional mechanisms (a nearly optimal lower bound for reputation-based approaches since Byzantine replicas can pretend to be correct, as discussed in Section IV-B) regardless of Byzantine behavior, while achieving significantly higher upper bounds.

Cost of Byzantine Behavior. In conventional mechanisms, Byzantine replicas simply wait for their turn to become leaders and disrupt consensus progress without contributing positively. SWLE penalizes leadership misbehavior by significant score reductions ($\alpha_1 = -n$). Once a Byzantine replica loses eligibility ($\text{score} < |\alpha_0| = 1$), its position may be replaced by correct ones. Regaining eligibility⁸ at correct replicas can only through: (a) having its valid proposals finalized (R3), (b) contributing as one of the first $2f+1$ valid votes in successful consensus rounds led by correct replicas (R4), or (c) waiting for R5 be triggered. Since (a) becomes unavailable after losing eligibility, Byzantine replicas must rely on (c) which has a very long cycle ($\geq 10n$ views), or (b) which provides only $(1/n)$ -point increases at 1 correct replica per view thus requiring sustained correct behavior across multiple views even with non-negative score constraint. This creates a trade-off for attackers (mainly after GST): they must contribute positively for a period (or wait a considerable period) before gaining the opportunity to disrupt it - the overall performance improves compared to scenarios where they can disrupt without such constraints.

VI. EVALUATION

Overview. We implemented SWLE as a protocol-independent Rust module containing all core functionalities and data structures (approximately 400 lines of code). The implementation relies on an extra cryptographic module (built primarily using `ed25519_dalek`) alongside standard library dependencies. The two modules are open-sourced on GitHub⁹ to ensure reproducibility of our experimental conclusions.

⁸Byzantine replicas may also achieve leadership via certain uneconomical strategies without first regaining eligibility at correct replicas (after GST). We provide a concrete example of such a strategy in [39].

⁹<https://github.com/BerserkRugal/SWLE>

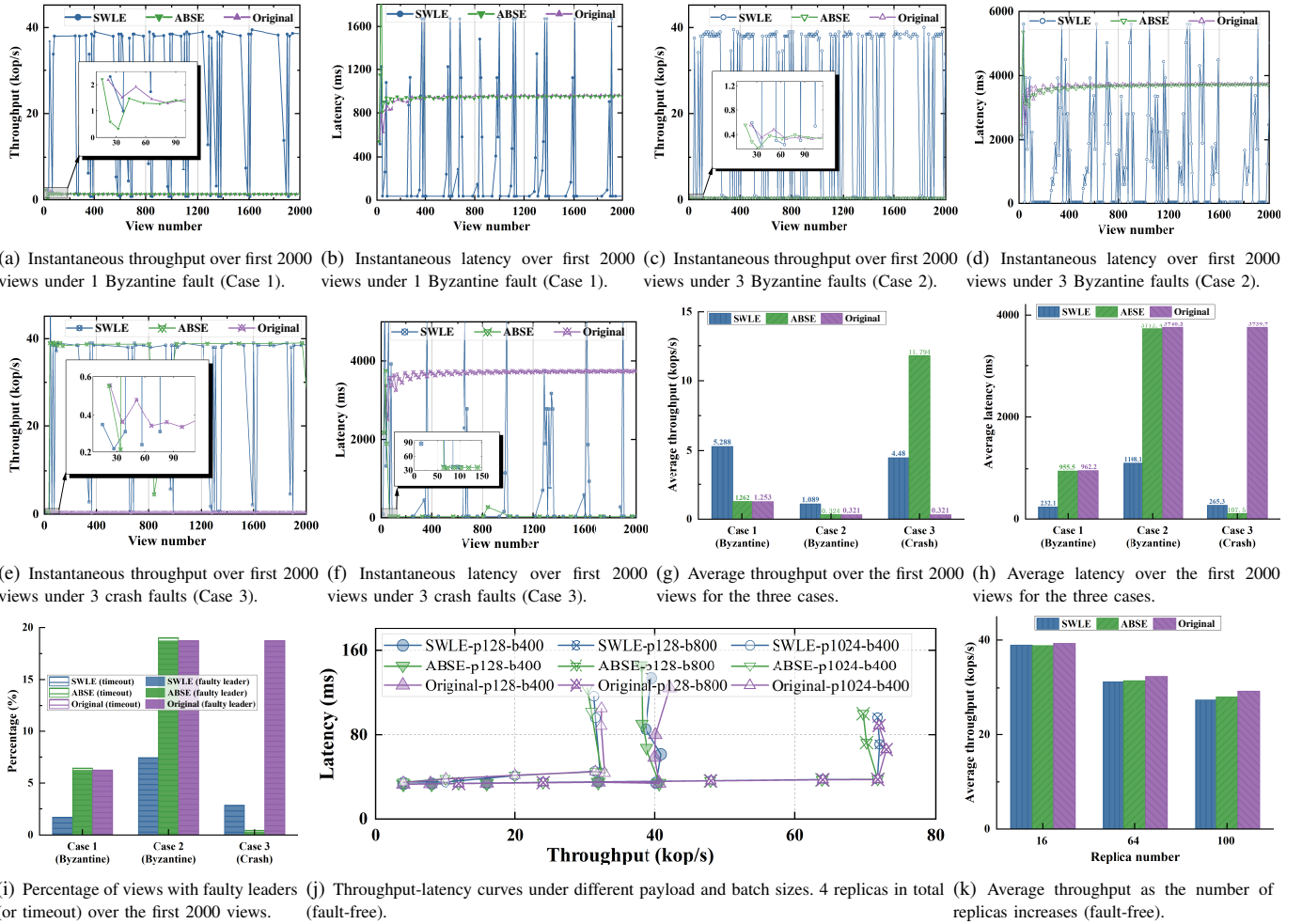


Fig. 1. Performance.

For evaluation, we integrated SWLE into HotStuff protocol [9] (non-chained version), requiring ~ 300 lines of code modifications. We compare SWLE against ABSE, a state-of-the-art reputation-based leader election optimization framework (using the default open-source module provided in the original work [18]), and HotStuff's default round-robin leader election mechanism. For brevity, we refer to these three approaches as "SWLE", "ABSE", and "Original" respectively in subsequent figures. Our experiments demonstrate that SWLE: (1) imposes higher misbehavior costs on Byzantine replicas to maintain their leadership eligibility and provides superior performance under common Byzantine fault scenarios, (2) addresses critical limitations in ABSE where recovery time after GST remains unbounded, and (3) has a minor impact on base protocol performance in fault-free scenarios (lightweight nature).

Setup. We use throughput (operations finalized per second) and latency (duration from client operation request generation to finalization at replicas) as our main performance metrics, categorized as: global average (average values from system startup) and instantaneous metrics (sampled over sliding windows, with periodic reporting). The metric module is also open-sourced to ensure experimental reproducibility, but note it is not protocol-independent, as it depends on block metadata and configuration parameters (e.g., sliding window size).

Experiments ran on 16 Alibaba Cloud ECS u1-c1m2.4xlarge instances across 4 different regions in northern China (Beijing, Zhangjiakou, Hohhot, and Ulanqab). Each machine provides 5Gbps bandwidth, 16 virtual CPUs on a 2.5GHz Intel Xeon Platinum scalable processor, 32GB memory, and runs Ubuntu 22.04 LTS. Replicas were distributed as equally as possible across these instances. Unless otherwise specified, batch size was 400 operations, operation size was 128 bytes, and timeout was 1.5 seconds per replica (note timeout configuration would influence observed performance under faults: shorter timeouts reduce degradation by faster leader rotation, while longer timeouts amplify performance gaps between mechanisms).

Performance under Faulty Replicas. We conducted experiments with a total of 16 replicas ($n=16$), prioritizing deployment of faulty replicas on machines with lower average message delays to maximize their advantage (and minimize their cost). Experiments were conducted under three fault scenarios: 1 Byzantine faulty replica (Case 1); 3 Byzantine faulty replica (Case 2); and 3 crash faulty replicas (Case 3). We note that experiments with 5 Byzantine/crash faulty replicas were also conducted but are omitted, as the performance variance significantly impacted graph readability, and the results showed similar trends to those observed under 1 or 3 faults.

Byzantine faults strategically build reputation through rapid

and valid voting (low-cost given their location advantage) then disrupt consensus (lead maliciously) when they gain leadership. Note in original HotStuff, no reputation mechanism exists, so Byzantine faults simply wait for their turn to become leaders without participating in voting. Crash faults discard all messages and remain unresponsive. Performance metrics were measured over the first 2000 views of system execution.

Case 1: As shown in Figs. 1(a)–1(b), SWLE maintains throughput at ~ 38 kop/s and latency at ~ 37 ms in most views, with Byzantine leaders being elected and disrupting consensus infrequently. In contrast, ABSE and the original mechanism exhibit severe degradation: initial throughput briefly peaks at ~ 2.2 kop/s (~ 600 ms latency) before subsequently stabilizing at ~ 1.27 kop/s (~ 950 ms latency)¹⁰. This persistent performance gap—visible across all sampling windows—indicates that Byzantine replicas can achieve stable election frequency, which suggest that both mechanisms provide weaker guarantees in leader election against common Byzantine behaviors compared to SWLE: as reflected in Fig. 1(i), SWLE experiences Byzantine leaders in $<2\%$ of views, while the other two mechanisms exceed 6% . Consequently, SWLE achieves ~ 5.3 kop/s average throughput (Fig. 1(g))— $\sim 4.2\times$ higher than baselines (~ 1.26 kop/s)—and ~ 232 ms average latency (Fig. 1(h)), $\sim 75\%$ lower than ~ 960 ms in ABSE/Original.

Notably, ABSE suffers an initial performance drop: Byzantine replicas exploit their faster responsiveness to dominate early voting, thereby preempting some correct replicas as consensus-promoting replicas per view. This leads to scenarios where no replicas can accumulate scores above the base reputation score thresholds (baseline) for leadership eligibility (score of Byzantine replicas are insufficient as well due to misleading) at $2f+1$ replicas during certain views in the initial period, resulting in ABSE’s conflict scenarios (timeout occurs and roll back to the original HotStuff). This is also evidenced in Fig. 1(i), as ABSE’s view timeout percentage exceeds the percentage of views led by Byzantine replicas. While recovery occurs quickly here, this exposes a critical limitation of ABSE: if adversaries consistently suppress correct replicas’ reputation accumulation at others pre-GST, recovery time becomes unbounded post-GST since the baseline scores increase with view counts in ABSE. SWLE avoids this by design, providing bounded post-GST recovery (which we have proved in theory). *Case 2 (similar to Case 1):* With increased Byzantine faults (Figs. 1(c)–1(d)), ABSE/Original degrade further, stabilizing at ~ 0.33 kop/s throughput and ~ 3.7 s latency. Though affected by Byzantine leaders in more views ($\sim 4\times$) compared to Case 1 (Fig. 1(i)), SWLE still maintains relatively fast response for most operations. Its faulty-leader rate $\sim 7.5\%$ remains $\sim 60\%$ lower than ABSE/Original and sustains ~ 1.1 kop/s average throughput— $\sim 3.3\times$ higher than the other two—and ~ 1.1 s latency (Fig. 1(g)–1(h)), $\sim 70\%$ lower than ~ 3.7 s.

¹⁰It is observed that ABSE performs slightly better than original mechanism (e.g., throughput $\sim 1\%$ higher) after stabilization. This occurs because under ABSE, Byzantine replicas contribute valid votes to accumulate reputation scores, but this isn’t the case under the original mechanism. Since they respond faster, consensus efficiency in rounds led by correct replicas is slightly higher.

Case 3: While SWLE isn’t specifically optimized for crash fault scenarios (Figs. 1(e)–1(f)) and therefore does not outperform ABSE, it is still superior to Original: optimum throughput reaches 38 kop/s (similar to ABSE), but SWLE sustains this optimum for fewer views (Original performs similarly to Case 2). As shown in Fig. 1(i), crash replicas become leaders in $\sim 2.8\%$ of SWLE views— $\sim 9\times$ more than ABSE—resulting in lower average throughput (~ 4.5 kop/s vs. ABSE’s ~ 11.3 kop/s) and higher latency (~ 265 ms vs. ~ 107 ms in Figs. 1(g)–1(h)). However, optimizing specifically for crash fault scenarios in BFT systems may inadvertently create vulnerabilities exploitable by Byzantine adversaries.

Base Performance (Fault-Free Case). We measured throughput and latency of HotStuff under three mechanisms in a setting commonly used for evaluating other BFT systems [9], [40] to test their performance characteristics (L-graphs). Experiments employed 4 replicas with two variable dimensions: (1) Operation (payload) sizes: 128 ("p128") and 1024 bytes ("p1024"), (2) Batch sizes: 400 ("b400") and 800 ("b800"). We incrementally increased client request rate until system saturation, with identical rate steps (under the same configuration) across mechanisms. As shown in Fig. 1(j), overall, all three mechanisms exhibit minimal peak throughput differences (e.g., ~ 72 kop/s at p128-b800 and differences within 1%). SWLE demonstrates slightly superior stability post-saturation compared to ABSE, though is slightly weaker than Original. Varying payload and batch sizes hardly cause changes in performance deviation between SWLE and Original.

Scalability (Fault-Free Case). We assess scalability by measuring average throughput at increasing replica counts ($n=16, 64, 100$). At $n=16$, SWLE achieves ~ 38.8 kop/s — comparable to ABSE but slightly lower than Original. At $n=64$, SWLE begins to underperform compared to ABSE, achieving ~ 31.3 kop/s — $\sim 1\%$ lower than ABSE and $\sim 4\%$ lower than Original. While these throughput gaps expand to $\sim 2.5\%$ and $\sim 7\%$ respectively when $n=100$. The widening gap stems from SWLE’s per-view overhead in candidate generation and leader determination across all replicas, making the mechanism somewhat sensitive to increases in replica numbers. This highlights a potential optimization direction for future work.

VII. CONCLUSIONS

We address limitations in existing reputation-based leader election frameworks for partially synchronous BFTs. We present a novel protocol-independent abstraction formalizing core properties for theoretical analysis and design. Building on this, we design SWLE, a novel reputation-based leader election mechanism that provides enhanced guarantees. We show, with a 16-server deployment, SWLE achieves superior performance to the state-of-the-art solution under common Byzantine faults, while maintaining efficiency in fault-free scenarios.

VIII. ACKNOWLEDGEMENT

This work is supported by the National Natural Science Foundation of China (NSFC) under Grant 62232002, Grant U2468205 and Grant 625B2022.

REFERENCES

- [1] L. Lamport, R. Shostak, and M. Pease, “The byzantine generals problem,” in *Concurrency: the works of leslie lamport*, 2019, pp. 203–226.
- [2] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.
- [3] M. Ben-Or, “Another advantage of free choice (extended abstract) completely asynchronous agreement protocols,” in *Proceedings of the 2nd Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 1983, pp. 27–30.
- [4] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.
- [5] L. Lamport, “Fast paxos,” *Distributed Computing*, vol. 19, no. 2, pp. 79–103, 2006.
- [6] M. Castro, B. Liskov *et al.*, “Practical byzantine fault tolerance,” in *OSDI*, vol. 99, no. 1999, 1999, pp. 173–186.
- [7] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, “Hyperledger fabric: a distributed operating system for permissioned blockchains,” in *Proceedings of the thirteenth EuroSys conference*, 2018.
- [8] G. G. Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, “Sbft: A scalable and decentralized trust infrastructure,” in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2019, pp. 568–580.
- [9] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, “Hotstuff: Bft consensus with linearity and responsiveness,” in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.
- [10] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin, “Sync hotstuff: Simple and practical synchronous state machine replication,” in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 106–118.
- [11] X. Liu, K. Feng, Z. Zhang, M. Li, X. Chen, W. Lai, and L. Zhu, “Dolphin: Efficient non-blocking consensus via concurrent block generation,” *IEEE Transactions on Mobile Computing (TMC)*, vol. 23, no. 12, pp. 11 824–11 838, 2024.
- [12] M. Kelkar, F. Zhang, S. Goldfeder, and A. Juels, “Order-fairness for byzantine consensus,” in *Advances in Cryptology—CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part III* 40. Springer, 2020, pp. 451–480.
- [13] M. Kelkar, S. Deb, S. Long, A. Juels, and S. Kannan, “Themis: Fast, strong order-fairness in byzantine consensus,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 475–489.
- [14] N. Alhaddad, S. Das, S. Duan, L. Ren, M. Varia, Z. Xiang, and H. Zhang, “Balanced byzantine reliable broadcast with near-optimal communication and improved computation,” in *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC)*, 2022, pp. 399–417.
- [15] L. Yang, S. J. Park, M. Alizadeh, S. Kannan, and D. Tse, “{DispersedLedger}::{High-Throughput} byzantine consensus on variable bandwidth networks,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 493–512.
- [16] G. Zhang and H.-A. Jacobsen, “Prosecutor: An efficient bft consensus algorithm with behavior-aware penalization against byzantine attacks,” in *Proceedings of the 22nd International Middleware Conference*, 2021, pp. 52–63.
- [17] G. Zhang, F. Pan, S. Tijanic, and H.-A. Jacobsen, “Prestigebft: Revolutionizing view changes in bft consensus algorithms with reputation mechanisms,” in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 1930–1943.
- [18] X. Liu, Z. Zhang, Z. Li, H. Yin, M. Li, J. Liu, M. Conti, and L. Zhu, “Abse: Adaptive baseline score-based election for leader-based bft systems,” *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 36, no. 8, pp. 1634–1650, 2025.
- [19] S. Cohen, R. Gelashvili, L. K. Kogias, Z. Li, D. Malkhi, A. Sonnino, and A. Spiegelman, “Be aware of your leaders,” in *International Conference on Financial Cryptography and Data Security (FC)*. Springer, 2022, pp. 279–295.
- [20] G. Tsimos, A. Kichidis, A. Sonnino, and L. Kokoris-Kogias, “Hammerhead: Leader reputation for dynamic scheduling,” in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2024, pp. 1377–1387.
- [21] L. Lamport, “Paxos made simple,” *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001), pp. 51–58, 2001.
- [22] P. Feldman and S. Micali, “An optimal probabilistic protocol for synchronous byzantine agreement,” *SIAM Journal on Computing*, vol. 26, no. 4, pp. 873–933, 1997.
- [23] S. Liu, M. Xu, Y. Zheng, Y. Zou, W. Qiu, G. Qu, and X. Cheng, “Partially synchronous bft consensus made practical in wireless networks,” in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.
- [24] Y. Gao, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, “Dumbo-ng: Fast asynchronous bft consensus with throughput-oblivious latency,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 1187–1201.
- [25] S. Duan, X. Wang, and H. Zhang, “Fin: Practical signature-free asynchronous common subset in constant time,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 815–829.
- [26] X. Liu, Z. Zhang, Z. Li, X. Lu, M. Li, L. Xu, M. Ao, and L. Zhu, “Group bft: Two-round bft protocols via replica grouping,” *IEEE Transactions on Dependable and Secure Computing (TDSC)*, vol. 22, no. 6, pp. 6309–6326, 2025.
- [27] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: Dag bft protocols made practical,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 2705–2718.
- [28] F. Borran and A. Schiper, “A leader-free byzantine consensus algorithm,” in *International Conference on Distributed Computing and Networking (ICDCN)*. Springer, 2010, pp. 67–78.
- [29] P.-L. Aublin, S. B. Mokhtar, and V. Quéma, “Rbft: Redundant byzantine fault tolerance,” in *2013 IEEE 33rd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2013, pp. 297–306.
- [30] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, “The honey badger of bft protocols,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 31–42.
- [31] T. Crain, C. Natoli, and V. Gramoli, “Red belly: A secure, fair and scalable open blockchain,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 466–483.
- [32] E. Buchman, “Tendermint: Byzantine fault tolerance in the age of blockchains,” Ph.D. dissertation, University of Guelph, 2016. [Online]. Available: <http://hdl.handle.net/10214/9769>
- [33] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” in *Concurrency: the Works of Leslie Lamport*, 2019, pp. 179–196.
- [34] J. Niu, X. Wen, G. Wu, S. Liu, J. Yu, and Y. Zhang, “Achilles: Efficient tee-assisted bft consensus via rollback resilient recovery,” in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, 2025, pp. 193–210.
- [35] H. Lyu, S. Xie, J. Niu, C. Feng, Y. Zhang, and I. Beschastnikh, “Ladon: High-performance multi-bft consensus via dynamic global ordering (eurosyst),” in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, pp. 226–242.
- [36] G. S. Veronese, M. Correia, A. N. Bessani, and L. C. Lung, “Spin one’s wheels? byzantine fault tolerance with a spinning primary,” in *2009 28th IEEE International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2009, pp. 135–144.
- [37] X. Sui, X. Wang, and S. Duan, “Signature-free atomic broadcast with optimal $o(n^2)$ messages and $o(1)$ expected time,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1547–1565.
- [38] J. Garay, A. Kiayias, and N. Leonardos, “The bitcoin backbone protocol: Analysis and applications,” *Journal of the ACM*, vol. 71, no. 4, pp. 1–49, 2024.
- [39] X. Liu, Z. Zhang, Z. Li, J. Sun, J. Liu, and P. Jiang, “Reputation-based leader election under partial synchrony: Towards a protocol-independent abstraction with enhanced guarantees,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.12409>
- [40] A. Bessani, J. Sousa, and E. E. Alchieri, “State machine replication for the masses with bft-smart,” in *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2014, pp. 355–362.